

SKILL-4 PROGRAMS

2520030331 ARNAV CHITIKENA

```
GNU nano 8.7.1 skill4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 256
#define MAX_TOKENS 50

typedef struct Token {
    char *value;
    struct Token *next;
} Token;

typedef struct ParseNode {
    char *value;
    struct ParseNode *left;
    struct ParseNode *right;
} ParseNode;

Token *createToken(char *value) {
    Token *token = malloc(sizeof(Token));
    if (token == NULL)
        exit(1);

    token->value = malloc(strlen(value) + 1);
    if (token->value == NULL)
        exit(1);

    strcpy(token->value, value);
    token->next = NULL;

    return token;
}

void addToken(Token **head, char *value) {
    Token *newToken = createToken(value);

    if (*head == NULL) {
        *head = newToken;
        return;
    }

    Token *temp = *head;
```

```
GNU nano 8.7.1 skill4.c
while (temp->next != NULL)
    temp = temp->next;

temp->next = newToken;
}

void tokenize(char *input, Token **head) {
    char buffer[MAX_INPUT];
    int i = 0;
    int j = 0;

    while (input[i] != '\0') {
        if (isspace((unsigned char)input[i]) ||
            input[i] == '|' ||
            input[i] == '&' ||
            input[i] == ';' ||
            input[i] == '<' ||
            input[i] == '>') {
            if (j > 0) {
                buffer[j] = '\0';
                addToken(head, buffer);
                j = 0;
            }

            if (isspace((unsigned char)input[i])) {
                buffer[0] = input[i];
                buffer[1] = '\0';
                addToken(head, buffer);
            }
            else {
                buffer[j++] = input[i];
            }
            i++;
        }
        if (j > 0) {
            buffer[j] = '\0';
            addToken(head, buffer);
        }
    }
}
```

```
arnav@LAPTOP-BN8992KC: ~ - + v
GNU nano 8.7.1 skill4.c

int validateTokens(Token *head) {
    Token *temp = head;

    if (head == NULL)
        return 0;

    while (temp != NULL) {
        if (strlen(temp->value) == 0)
            return 0;

        temp = temp->next;
    }

    return 1;
}

void displayTokens(Token *head) {
    Token *temp = head;
    int count = 1;

    printf("\nTokens:\n");

    while (temp != NULL) {
        printf("Token %d: %s\n", count++, temp->value);
        temp = temp->next;
    }
}

ParseNode *createNode(char *value) {
    ParseNode *node = malloc(sizeof(ParseNode));

    if (node == NULL)
        exit(1);

    node->value = malloc(strlen(value) + 1);

    if (node->value == NULL)
        exit(1);

    strcpy(node->value, value);

    node->left = NULL;
}
```

```
arnav@LAPTOP-BN8992KC: ~ - + v
GNU nano 8.7.1 skill4.c

    return node;
}

ParseNode *buildParseTree(Token *head) {
    ParseNode *root;
    ParseNode *current;

    if (head == NULL)
        return NULL;

    root = createNode(head->value);
    current = root;
    head = head->next;

    while (head != NULL) {
        current->right = createNode(head->value);
        current = current->right;
        head = head->next;
    }

    return root;
}

int validateSyntax(Token *head) {
    Token *temp = head;

    while (temp != NULL) {
        if ((strcmp(temp->value, "|") == 0 ||
             strcmp(temp->value, "&&") == 0 ||
             strcmp(temp->value, ",") == 0) &&
            (temp->next == NULL ||
             strcmp(temp->next->value, "|") == 0 ||
             strcmp(temp->next->value, "&&") == 0 ||
             strcmp(temp->next->value, ",") == 0)) {
            return 0;
        }

        temp = temp->next;
    }

    return 1;
}
```

```
arnav@LAPTOP-BN8992KC: ~  
GNU nano 8.7.1 skill4.c  
    return 1;  
}  
  
void displayParseTree(ParseNode *root, int level) {  
    if (root == NULL)  
        return;  
  
    for (int i = 0; i < level; i++)  
        printf(" ");  
  
    printf("%s\n", root->value);  
  
    displayParseTree(root->right, level + 1);  
}  
  
void freeTokens(Token *head) {  
    Token *temp;  
  
    while (head != NULL) {  
        temp = head;  
        head = head->next;  
        free(temp->value);  
        free(temp);  
    }  
}  
  
void freeParseTree(ParseNode *root) {  
    if (root == NULL)  
        return;  
  
    freeParseTree(root->left);  
    freeParseTree(root->right);  
    free(root->value);  
    free(root);  
}  
  
int main() {  
    char input[MAX_INPUT];  
    Token *tokens = NULL;  
    ParseNode *tree = NULL;  
  
    printf("Enter command: ");  
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M Set Mark   ^I To Bracket  
    ^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^E Redo       ^G Copy       ^B Where Was
```

```
arnav@LAPTOP-BN8992KC: ~  
GNU nano 8.7.1 skill4.c  
    Token *tokens = NULL;  
    ParseNode *tree = NULL;  
  
    printf("Enter command: ");  
    fgets(input, sizeof(input), stdin);  
  
    input[strcspn(input, "\n")] = '\0';  
  
    if (strlen(input) == 0) {  
        printf("Empty command\n");  
        return 0;  
    }  
  
    tokenize(input, &tokens);  
  
    if (!validateTokens(tokens)) {  
        printf("Invalid token stream\n");  
        freeTokens(tokens);  
        return 0;  
    }  
  
    displayTokens(tokens);  
  
    if (!validateSyntax(tokens)) {  
        printf("Syntax error\n");  
        freeTokens(tokens);  
        return 0;  
    }  
  
    tree = buildParseTree(tokens);  
  
    printf("\nParse Tree:\n");  
    displayParseTree(tree, 0);  
  
    printf("\nSyntax is valid\n");  
    printf("Execution structure generated successfully\n");  
  
    freeTokens(tokens);  
    freeParseTree(tree);  
  
    return 0;  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M Set Mark   ^I To Bracket  
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^E Redo       ^G Copy       ^B Where Was
```

OUTPUT

```
arnav@LAPTOP-BN8992KC:~/OS$ nano skill4.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc skill4.c -o skill4
./skill4
Enter command: echo hello world

Tokens:
Token 1: echo
Token 2: hello
Token 3: world

Parse Tree:
echo
  hello
    world

Syntax is valid
Execution structure generated successfully
```